Programação Orientada a Objetos

Refinamento de Interface e Diagramas UML

Professor: Pedro Montoaneli
pedromontoaneli@teiacoltec.org

Sumário

- Refinamento de Interface
  - O Qualificador const
  - Métodos e Atributos const
  - Sobrecarga e Argumento Padrão
  - Exemplo
- Diagramas UML
  - Estrutura básica
  - Notação das relações
- Exercícios

O Qualificador const

- Atribuir o `const` a um parâmetro significa informar ao compilador que aquele parâmetro não será alterado.
- No caso de variáveis, o `const` cria uma constante.
- No caso de passagem de parâmetros por referência, a função pode ler o conteúdo, mas sem copiá-lo nem modifica-lo..

```
const double PI = 3.14159;
void imprimir(const string& texto) {
     cout << texto << endl;
}
```

Métodos e Atributos const

- Métodos `const` não alteram o objeto. O qualificador deve vir depois dos parâmetros.
- Um objeto recebido por `const` referência só aceita chamadas de métodos `const`.

```
string getNome() const {
        return nome;
}
```

- Em funções que recebem objetos, eles devem ser passados como `const` referência.

```
string apresentar(const Disciplina& d){
        cout << d.getNome() << endl;
}
```

Sobrecarga e Argumentos Padrão

- A sobrecarga consiste em usar o mesmo nome para métodos diferentes. A diferenciação entre eles ocorre na lista de parâmetros.

```
void imprimir(int x){
        cout << “Inteiro: ” << x << endl;
}
```

```
void imprimir(float x){
        cout << “Decimal: ” << x << endl;
}
```

- Caso os métodos tenham o mesmo corpo, usa-se o argumento padrão.

```
void mensagem(string s = “Bem vindos”){
        cout << s<< endl;
}
```

```
void mensagem(string s){
        cout << s<< endl;
}
```

Exemplo: Classe Disciplina refinada

```
class Disciplina {
private:
    string nomeDisciplina;
    string nomeProfessor;

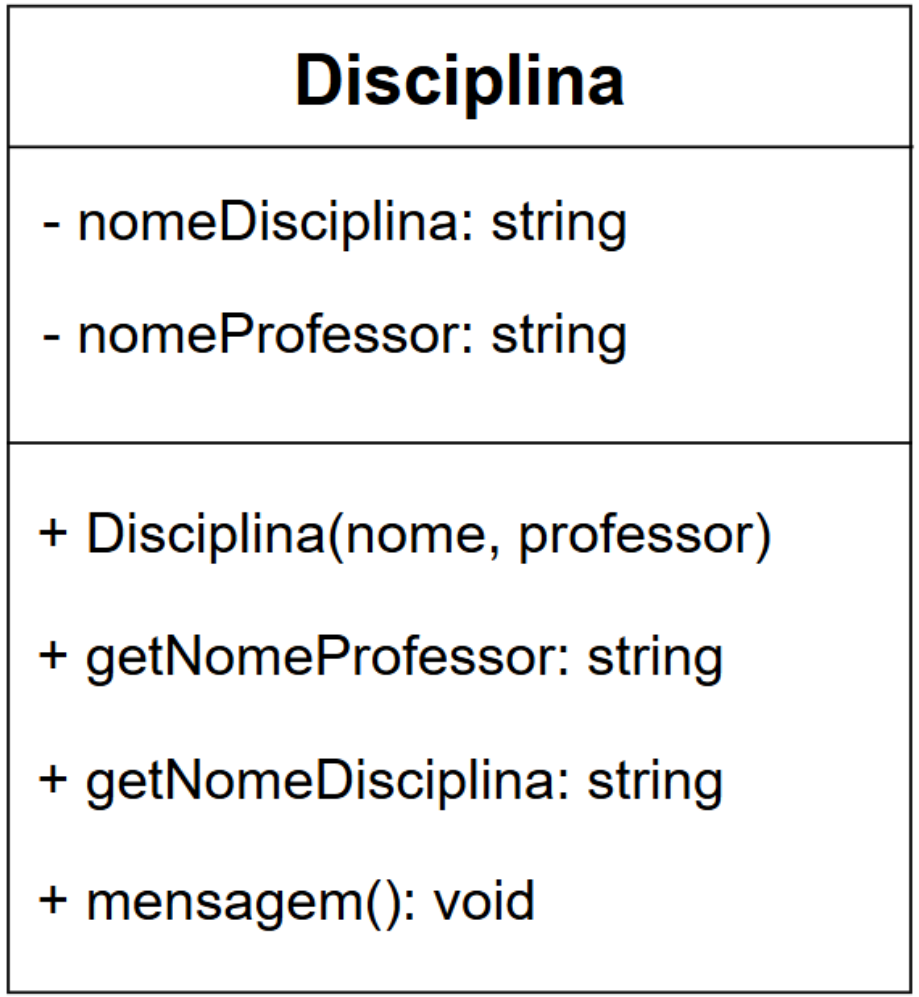
public:
    Disciplina(string nome, string professor) {
        this->nomeDisciplina = nome;
        this->nomeProfessor = professor;
    }

    string getNomeDisciplina() const { return nomeDisciplina; }
    string getNomeProfessor()  const { return nomeProfessor;  }

    void mensagem(string saudacao = "Bem vindos") const {
        cout << saudacao << " a " << nomeDisciplina << "!" << endl;
        cout << "Professor: " << nomeProfessor << endl;
    }
};
```

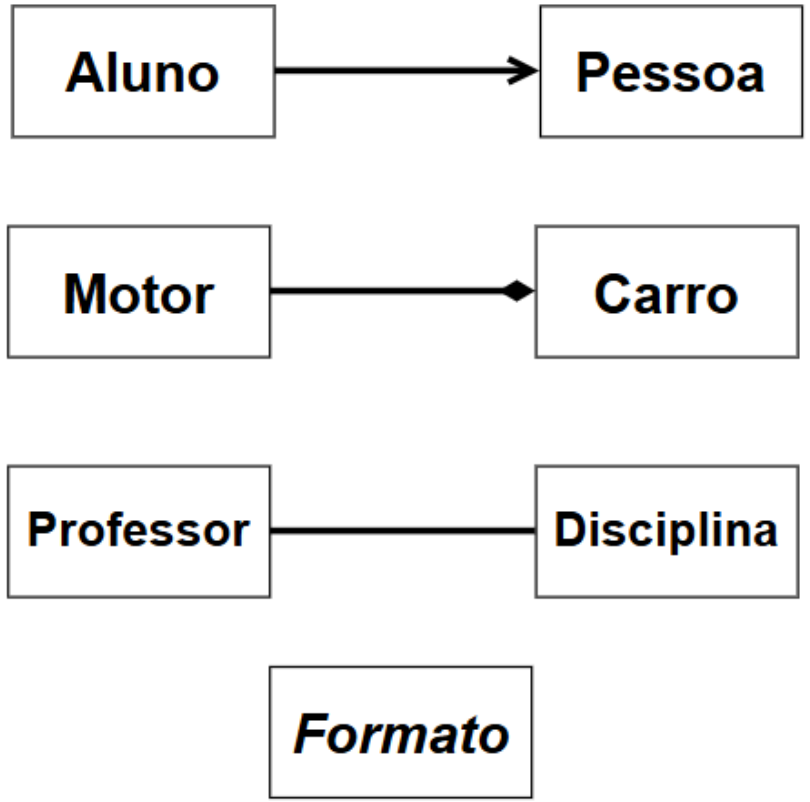
Estrutura Básica

- O UML é uma estrutura de modelagem de sistemas (*Universal Modeling Language*).
- A estrutura de uma classe é uma caixa com 3 partições: Nome, Atributos e Métodos.
- Marcadores:
  - `+ public`
  - `- private`
  - `# protected`
- Cada item mostra o tipo do atributo ou do retorno.



Notação das Relações

- Herança (é um): linha com triângulo vazado.
- Composição (tem um): losango preenchido apontando para o “todo”.
- Associação: linha simples.
- Classe abstrata: nome em itálico.



Exercícios

Exercício 1

Crie uma classe `Livro` com atributos privados `titulo`, `autor (string)` e `ano (int)`. Construtor com os três valores. Forneça getters, todos marcados como `const`. Sobrecarregue um método `descricao`, uma versão sem parâmetro que imprime título e autor, e outra com um `bool` que, quando verdadeiro, imprime também o ano. Escreva uma função livre `void catalogar()` que recebe o livro por `const` referência e imprime seus dados usando os getters. No `main`, crie dois livros e chame catalogar nos dois.

Exercício 2

Faça o diagrama UML da classe Livro.